

DSA Problem Explanation

Absolutely. This problem is mainly a **combinatorics + modular arithmetic** problem. The key is to notice that once the first character is fixed as **1**, the remaining positions can be arranged independently.

1. What is the problem asking?

We have:

- **N** zeros
- **M** ones

So the total string length is:

$$N + M$$

We need to count **unique permutations** of these characters that **start with 1**.

For example:

$$N = 1, M = 1$$

Characters:

0 1

Unique permutations:

01
10

Only:

10

starts with 1.

Answer = 1.

2. Key observation

Suppose we have:

N zeros
M ones

and the string **must start with 1**.

We can simply fix one 1 at the beginning:

1 _ _ _ _ ... _

Now what remains?

- **N** zeros
- **M - 1** ones

Total remaining positions:

$$N + M - 1$$

So the problem becomes:

How many unique ways can we arrange **N** zeros and **M-1** ones?

3. Combinations

There are:

$$N + M - 1$$

remaining positions.

We need to choose which positions contain the N zeros.

Therefore:

$$\text{Answer} = \binom{N + M - 1}{N}$$

We could also choose positions for the remaining ones:

$$\binom{N + M - 1}{M - 1}$$

Both are equal.

4. Let's build the intuition step-by-step

Brute force idea

Imagine generating **every permutation** of the string.

For:

```
00111
```

we could generate:

```
00111
01011
01101
01110
10011
10101
10110
11001
11010
11100
```

Then count the ones beginning with 1.

But this is terrible.

The number of permutations can be huge.

Even though the characters repeat, generating permutations is essentially factorial-sized.

So we need mathematics.

5. Better observation: Fix the first character

Because the answer only wants strings starting with 1, don't generate strings that start with 0.

Simply put one 1 at the beginning:

```
1
```

Suppose:

```
N = 2
M = 3
```

We have:

```
00 111
```

Fix one 1:

```
1 _ _ _ _
```

Remaining:

```
00 11
```

Now we need to arrange:

```
0 0 1 1
```

The unique arrangements are:

```
0011
0101
0110
1001
1010
1100
```

Putting the fixed 1 back:

```
10011
10101
10110
11001
11010
11100
```

Exactly 6.

6. Why is it $C(N+M-1, N)$?

For:

```
N = 2
M = 3
```

after fixing the first 1:

```
1 _ _ _ _
```

we have 4 positions.

We need to place two 0s.

Choose any 2 of these 4 positions:

$$\begin{aligned} & \binom{4}{2} \\ &= \frac{4!}{2!2!} \\ &= 6 \end{aligned}$$

Therefore:

```
Answer = C(4,2) = 6
```

7. But N and M can be 1000

We have:

```
1 ≤ N, M ≤ 1000
```

So:

$$N + M - 1 \leq 1999$$

We need to calculate:

$$\binom{N+M-1}{N} \mod (10^9 + 7)$$

We cannot calculate factorials normally because factorial numbers become enormous.

For example:

$$1000!$$

is far too large for normal integer types.

So we use **modular arithmetic**.

8. Formula for nCr

Normally:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Under modulo:

$$\binom{n}{r} = n! \times (r!)^{-1} \times ((n-r)!)^{-1} \pmod{MOD}$$

where:

$$MOD = 10^9 + 7$$

We need the **modular inverse** of factorials.

Using Fermat's Little Theorem:

$$a^{-1} \equiv a^{MOD-2} \pmod{MOD}$$

So:

$$invFact[r] = fact[r]^{MOD-2}$$

9. Precompute factorials

Since the maximum value is only:

1999

we can precompute:

```
fact[0]
fact[1]
fact[2]
...
fact[1999]
```

where:

```
fact[i] = i! % MOD
```

Then:

$$C(n, r) = fact[n] \times invFact[r] \times invFact[n-r] \pmod{MOD}$$

10. C++ Code

```
C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const ll MOD = 1e9 + 7;
const int MAXN = 2005;

// Fast exponentiation
ll power(ll a, ll b) {
    ll ans = 1;

    while (b > 0) {
        if (b & 1) {
            ans = (ans * a) % MOD;
        }

        a = (a * a) % MOD;
        b >>= 1;
    }

    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // Precompute factorials
    vector<ll> fact(MAXN);
    vector<ll> invFact(MAXN);

    fact[0] = 1;

    for (int i = 1; i < MAXN; i++) {
        fact[i] = (fact[i-1] * i) % MOD;
    }

    // Inverse factorial
    invFact[MAXN-1] = power(fact[MAXN-1], MOD-2);

    for (int i = MAXN-2; i >= 0; i--) {
        invFact[i] = (invFact[i+1] * (i+1)) % MOD;
    }
}
```

```
int T;
cin >> T;

while (T--) {
    int N, M;
    cin >> N >> M;

    int n = N + M - 1;
    int r = N;

    ll ans = fact[n];

    ans = (ans * invFact[r]) % MOD;
    ans = (ans * invFact[n - r]) % MOD;

    cout << ans << '\n';
}

return 0;
}
```

11. Understanding the important part of the code

The main calculation is:

```
C++
int n = N + M - 1;
int r = N;
```

Why?

Because after fixing the first 1:

```
1 _ _ _ _ ...
```

there are:

$$N + M - 1$$

positions left.

We need to put N zeros into those positions.

Therefore:

```
C++
C(N + M - 1, N)
```

Then:

```
C++
ll ans = fact[n];

ans = (ans * invFact[r]) % MOD;
ans = (ans * invFact[n - r]) % MOD;
```

This implements:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

as:

$$n! \times (r!)^{-1} \times ((n-r)!)^{-1}$$

12. Why calculate inverse factorial from the last element?

This is an important trick.

We first calculate:

```
C++
invFact[MAXN - 1]
```

using:

```
C++
invFact[MAXN - 1] =
    power(fact[MAXN - 1], MOD - 2);
```

Then we can calculate all previous values:

```
C++
invFact[i] = invFact[i + 1] * (i + 1) % MOD;
```

Because:

$$(i + 1)! = (i + 1) \times i!$$

Therefore:

$$\frac{1}{i!} = \frac{i + 1}{(i + 1)!}$$

So:

$$invFact[i] = invFact[i + 1] \times (i + 1)$$

This lets us calculate **all inverse factorials in $O(MAXN)$** instead of doing modular exponentiation for every factorial.

13. Complete dry run

Sample:

```
2
1 1
2 3
```

Test case 1

```
N = 1
M = 1
```

We need strings containing:

```
0 1
```

and starting with 1.

Fix 1:

```
1 _
```

Remaining:

```
0
```

Only:

```
10
```

Mathematically:

$$\begin{aligned} C(N + M - 1, N) \\ &= C(1 + 1 - 1, 1) \\ &= C(1, 1) \\ &= 1 \end{aligned}$$

Output:

```
1
```

Test case 2

```
N = 2
M = 3
```

We have:

```
00 111
```

Fix one 1 at the beginning:

```
1 _ _ _ _
```

Remaining:

```
00 11
```

There are:

$$2 + 3 - 1 = 4$$

positions.

We need to choose 2 positions for zeros:

$$C(4, 2)$$

Calculate:

$$\begin{aligned} C(4, 2) &= \frac{4!}{2!2!} \\ &= \frac{24}{4} \\ &= 6 \end{aligned}$$

The six valid strings are:

```
10011
10101
10110
11001
11010
11100
```

Output:

```
6
```

Therefore final output:

```
1
6
```

14. Complexity

Let:

$$MAXN = N + M \leq 2000$$

Precomputation

Factorials:

$$O(MAXN)$$

Inverse factorials:

$$O(MAXN)$$

Each test case

Calculating nCr takes:

$$O(1)$$

because factorials are already precomputed.

For T test cases:

$$O(T)$$

after precomputation.

Overall

$$O(MAXN + T)$$

Since $MAXN \approx 2000$, this is extremely fast.

Space

We store:

```
fact[]
invFact[]
```

Therefore:

$$O(MAXN)$$

The main idea to remember

Whenever you see:

N zeros + M ones, count unique permutations starting with 1

immediately think:

```
Fix one 1 at the beginning
↓
Remaining: N zeros + (M-1) ones
↓
Total remaining positions = N + M - 1
↓
Choose positions for N zeros
↓
Answer = C(N + M - 1, N)
```

So the entire problem boils down to:

$$\text{Answer} = \binom{N + M - 1}{N} \pmod{10^9 + 7}$$

This is the key observation that turns the problem from permutation generation into an **O(1) query after O(2000) preprocessing**.

ChatGPT can make mistakes. Check important info.